Guía del
K++ Bootstrap Studio

El mini-IDE para compilar, probar y desensamblar programas K++

Proyecto K++ · Esequiel Pena

Octubre de 2026

Índice

1. ¿Qué es el Bootstrap Studio?
2. Requisitos
3. Instalación y configuración
4. Tour de la interfaz
5. Tu primera compilación
6. Cómo testear los ejemplos del proyecto
7. Leyendo el desensamblado
8. Archivos generados: del `.kpp` al `.bin` de KolibriOS
9. Solución de problemas
10. Limitaciones actuales

1. ¿Qué es el Bootstrap Studio?

El **K++ Bootstrap Studio** (`kpp_bootstrap_studio.py`) es un mini-IDE gráfico escrito en Python con Tkinter, pensado para trabajar con el compilador K++ sin salir de una sola ventana. Su flujo de trabajo es:

```
EDITOR  →  K++ (kpp)  →  RAW / BIN  →  DESENSAMBLADO (objdump i386)
   ↑                                              ↓
   └──────────────── editas, recompilas ──────────┘
```

Con él puedes escribir o abrir un programa `.kpp` , compilarlo, ver la salida del compilador (incluido el AST), inspeccionar el código máquina x86 generado y llevarte el ejecutable a KolibriOS. Es la herramienta recomendada para **testear los ejemplos** de la suite L01–L16.

2. Requisitos

Requisito	Para qué	Cómo verificarlo
Linux (Debian / MX Linux o similar)	Entorno de desarrollo del proyecto	`uname -a`
Python 3	Ejecuta el Studio	`python3 --version`
Tkinter (`python3-tk`)	La interfaz gráfica	`python3 -c "import tkinter"`
Compilador C++17 + CMake ≥ 3.16	Construir `bin/kpp`	`g++ --version; cmake --version`
`objdump` (binutils)	La pestaña de desensamblado	`objdump --version`
`dd` (coreutils)	Separar la cabecera del binario	`dd --version`

> En Debian/MX Linux, si falta algo: `sudo apt install python3-tk build-essential cmake binutils` .

3. Instalación y configuración

Paso 1 — Clona el repositorio

```
git clone https://github.com/saykol31/KPP.git
cd KPP
```

Paso 2 — Compila el compilador K++

```
cmake -S . -B build
cmake --build build -j"$(nproc)"
```

Esto genera el ejecutable `bin/kpp`, que es lo que el Studio invoca por debajo.

Paso 3 — Ajusta la ruta en el Studio (¡imprescindible!)

Abre `kpp_bootstrap_studio.py` y localiza la sección `CONFIGURACIÓN` al inicio del archivo:

```
KPP_ROOT = Path(
    "/home/saykol/C/KPP"
)
```

Cámbiala por la ruta donde **tú** clonaste el repositorio, por ejemplo:

```
KPP_ROOT = Path(
    "/home/tu_usuario/KPP"
)
```

De esta ruta el Studio deriva todo lo demás: `bin/kpp`, la carpeta de ejemplos `projects/` y la carpeta de salida `Output/`.

Paso 4 — Lanza el Studio

```
python3 kpp_bootstrap_studio.py
```

> **El error más común:** si al compilar aparece `[ERROR] No existe K++: .../bin/kpp`, es que el Paso 2 no se completó o el Paso 3 apunta a la carpeta equivocada. Revisa ambos antes de seguir.

4. Tour de la interfaz

La ventana se divide en tres zonas:

Barra superior de herramientas

Botón	Qué hace
🔨 **COMPILAR**	Compila el archivo actual con `bin/kpp` y muestra la salida.
▶ **RUN**	Pipeline completo: compila → genera RAW/BIN → extrae el payload → lo desensambla con objdump.
🐞 **DEBUG**	Reservado: el depurador llegará después del bootstrap (ver §10).
💾 **GUARDAR**	Guarda el contenido del editor en el archivo actual.
📂 **ABRIR**	Abre un archivo `.kpp` (el diálogo inicia en la carpeta `projects/`).

Panel izquierdo: editor K++ SOURCE

Editor con números de línea y resaltado básico. Al abrir, carga un programa mínimo de ejemplo. La barra inferior muestra `Ln / Col` (línea y columna del cursor) y el indicador de estado (`LISTO`, `COMPILANDO...`, etc.).

Panel derecho: pestañas de resultados

Pestaña	Contenido
K++ OUTPUT	Salida estándar del compilador: cabecera `K++ COMPILER`, el AST del programa, errores de léxico/sintaxis/semántica y confirmaciones de los escritores de binario.
DISASSEMBLY	Desensamblado del código máquina generado (objdump, CPU i386, sintaxis Intel). Conmutadores **CODE** / **DATA** / **RAW** para cambiar la vista.

Barra inferior: SISTEMA / NOTIFICACIONES

Registro de eventos de la sesión: comandos ejecutados, advertencias y el veredicto final de cada compilación. Los estados que verás con más frecuencia:

Estado	Significado

`RUN OK`	El pipeline completo terminó bien.
`BOOTSTRAP OK`	Se generaron RAW y BIN correctamente.
`ERROR`	Falló la compilación (revisa la pestaña K++ OUTPUT).
`DISASM ERROR`	El binario se generó pero el desensamblado falló.
`BIN INVÁLIDO`	El archivo generado no tiene el formato esperado.
`SALIDA INCOMPLETA`	Faltó alguno de los artefactos esperados.

5. Tu primera compilación

1. Abre el Studio. El editor ya trae un programa mínimo:

```
int main() {
    int x = 10;
    return x;
}
```

2. Pulsa **▶ RUN**.

3. Observa la pestaña **K++ OUTPUT**: verás la cabecera del compilador, el AST (`Program → Function: main`) y al final `✔ K++ KolibriOS executable OK` .

4. Cambia a la pestaña **DISASSEMBLY**: ahí está tu programa traducido a ensamblador x86 de 32 bits.

5. En **SISTEMA / NOTIFICACIONES** deberías ver `RUN OK` y `BOOTSTRAP OK` .

La diferencia entre los dos botones principales:

- 🔨 **COMPILAR** solo ejecuta `./bin/kpp archivo.kpp` y te muestra la salida. Ideal para iterar rápido mientras escribes.
- ▶ **RUN** hace todo el pipeline: compila, genera `.raw` y `.bin`, recorta la cabecera de 36 bytes con `dd` y desensambla el payload con `objdump -m i386` en sintaxis Intel. Es lo que quieres para testear ejemplos.

6. Cómo testear los ejemplos del proyecto

El repositorio trae dos colecciones de programas de prueba:

- `projects/` — ejemplos generales (es donde el diálogo **ABRIR** empieza).
- `compiler/kpp/` — la suite oficial de pruebas **L01–L16** (`test_l01_tipos.kpp` , `test_l02_variables.kpp` , …).

Ruta de testeo sugerida (de fácil a avanzado)

#	Ejercicio	Qué compruebas
1	`test_l01_tipos.kpp`	Tipos `int` y `string`, `print` / `print_int`
2	`test_l02_variables.kpp`	Declaración, inicialización y suma de variables
3	`test_l06_impresion.kpp`	Impresión de literales de texto
4	`test_l04_funciones.kpp`	Definición, llamada y `return`
5	`test_l05_parametros.kpp`	Parámetros y argumentos
6	`test_l10_if_else_audit_20261009.kpp`	Condicionales `if` / `else if` / `else`
7	`test_l11_while_audit_20261009.kpp`	Bucles `while`
8	`kpp-l09-operadores-audit-20261009.kpp`	Operadores relacionales y lógicos (==, &&, `||`, `!`)
9	`test_l07_struct.kpp` / `test_l08_enum.kpp`	Declaración de `struct` y `enum`
10	`test_l14_errores.kpp`	Cómo responde el compilador ante errores (descomenta cada bloque `/* */` para probar)

Procedimiento para cada ejemplo

1. Pulsa 📂 **ABRIR** y navega al archivo (para la suite L01–L16, sube un nivel desde `projects/` hasta `compiler/kpp/`).

2. Pulsa ▶ **RUN**.

3. En **K++ OUTPUT** verifica que el AST refleje tu programa (funciones, variables, estructuras) y que no haya errores.

4. En **DISASSEMBLY** comprueba que se generó código x86 coherente (ver §7).

5. En **SISTEMA / NOTIFICACIONES** confirma `RUN OK` + `BOOTSTRAP OK`.

> **Caso L14 (errores):** ese archivo trae tres bloques comentados, cada uno con un error distinto (falta `;`, variable no declarada, tipos incompatibles). Descomenta uno, pulsa RUN y observa el mensaje del compilador; vuelve a comentarlo antes de probar el siguiente.

7. Leyendo el desensamblado

La pestaña **DISASSEMBLY** muestra la salida de `objdump` sobre el payload del binario (el archivo `.bin` menos sus 36 bytes de cabecera). La cabecera de la pestaña indica el archivo, `CPU: i386` y `Syntax: Intel`.

Los tres conmutadores de vista:

Vista	Muestra	Úsala para
CODE	El código máquina como instrucciones ensamblador	Verificar que tus funciones generan el x86 esperado (`call`, `ret`, accesos a pila…)
DATA	La zona de datos del binario	Localizar literales de texto y constantes
RAW	Volcado crudo del payload	Inspección byte a byte cuando algo no cuadra

> **Para qué sirve en la práctica:** si un ejemplo compila pero sospechas que el binario hace algo raro, el desensamblado te deja auditar lo que realmente se generó, instrucción por instrucción, antes de llevarlo a KolibriOS. También hay botones para **copiar** y **limpiar** el contenido de cada pestaña.

8. Archivos generados: del `.kpp` al `.bin` de KolibriOS

Cada compilación exitosa deja dos artefactos en la carpeta `Output/`, nombrados según tu archivo fuente (por ejemplo, `hola.kpp` → `hola.raw` y `hola.bin`):

Archivo	Qué es
`.raw`	Artefacto intermedio del pipeline K++.
`.bin`	**Ejecutable nativo de KolibriOS** (x86 de 32 bits, con cabecera de 36 bytes).

Para probarlo de verdad: copia el `.bin` a tu instalación o imagen de KolibriOS y ejecútalo allí. Los programas de ejemplo usan `print` y `print_int`, implementados en el runtime de K++ para KolibriOS, así que verás su salida en pantalla.

9. Solución de problemas

Síntoma	Causa probable	Solución
`[ERROR] No existe K++: .../bin/kpp`	El compilador no está construido o KPP_ROOT apunta mal	Repite los pasos 2 y 3 del §3
El diálogo ABRIR no muestra tus ejemplos	Apunta a otro `projects/`	Verifica KPP_ROOT; navega manualmente a tu carpeta
`DISASM ERROR` tras compilar bien	Falta `objdump` o `dd`	Instala `binutils` y `coreutils`
La ventana no abre: `no module named 'tkinter'`	Falta el binding gráfico de Python	`sudo apt install python3-tk`
Error del compilador en K++ OUTPUT	Error en tu programa `.kpp`	Lee el mensaje (suele ser un `;` olvidado o variable no declarada) y corrige en el editor
`BIN INVÁLIDO` / `SALIDA INCOMPLETA`	El pipeline no generó todos los artefactos	Revisa K++ OUTPUT buscando el primer error; prueba con un ejemplo conocido bueno (L01)

10. Limitaciones actuales

- El botón 🐞 **DEBUG** es un marcador de posición: el depurador llegará después del bootstrap. Por ahora, la verificación es `SOURCE → KPP → RAW/BIN`.
- El Studio está pensado para Linux; las rutas y herramientas (`dd`, `objdump`) asumen ese entorno.
- `KPP_ROOT` está hardcodeado: cada persona que clone el repo debe ajustarlo (§3, paso 3).
- El lenguaje que puedes testear es el subconjunto Stage 0 documentado en el *Manual de Programación del Lenguaje K++* (ver limitaciones allí: sin `* / %`, sin acceso a campos de `struct` con `.`, etc.).

Guía elaborada a partir del código fuente real de `kpp_bootstrap_studio.py` (repositorio `github.com/saykol31/KPP`). Complementa al *Manual de Programación del Lenguaje K++*.